

บทที่ 2

ทฤษฎีและเทคโนโลยีพื้นฐาน

ในส่วนของการใช้งานนั้นจะแบ่งออกเป็น 2 ส่วนใหญ่ๆ คือ ส่วนที่ทำงานบนเครื่องคอมพิวเตอร์ และโทรศัพท์เคลื่อนที่ ดังนั้นจึงต้องมีเครื่องมือที่ใช้ในการพัฒนาต่างกัน มีดังนี้คือ 1) สคริปต์จาวาที่ใช้สำหรับพัฒนาเว็บแอปพลิเคชันบนเครื่องคอมพิวเตอร์ (JSP) 2) โปรแกรมจาวาขนาดเล็กที่สามารถถูกรันบนเบราว์เซอร์ของเครื่องคอมพิวเตอร์ได้ (จาวาแอปเพล็ต Java Applet) และ 3) ภาษาจาวาที่ใช้พัฒนาแอปพลิเคชันบนโทรศัพท์เคลื่อนที่ (J2ME)

2.1 JSP

2.1.1 Java Servlet

Java Servlet คือ “โปรแกรม” ที่เขียนขึ้นมาเป็นคลาสของภาษาจาวาเพื่อทำงานทางฝั่งเซิร์ฟเวอร์ ทำนองเดียวกับ ASP หรือ PHP ซึ่งรันทางฝั่งเซิร์ฟเวอร์เช่นกัน การนำจาวาเซิร์ฟเล็ตมาใช้ประโยชน์จึงไม่แตกต่างกับ ASP หรือ PHP คือ ใช้สำหรับเขียนโปรแกรมในลักษณะเดียวกับ CGI เพื่อทำหน้าที่อ่านข้อมูลที่รับมาจากผู้ชมเว็บไซต์ แล้วเอาข้อมูลนั้นมาประมวลผล จากนั้นจึงส่งผลลัพธ์กลับไปให้แก่ผู้ชมเว็บนั่นเอง

เนื่องจากจาวาเซิร์ฟเล็ตมีจุดเด่นที่สำคัญมากมาย เช่น มีประสิทธิภาพและความเร็วสูงในการทำงาน, สามารถปรับปรุงแก้ไขและพัฒนาได้ง่ายเพราะใช้ภาษาจาวาซึ่งเป็นภาษาเชิงวัตถุ (Object-Oriented Language) ในการพัฒนา เป็นต้น จึงเป็นอีกทางเลือกหนึ่งในการพัฒนา CGI และได้รับความนิยมนำมาพัฒนาเว็บแอปพลิเคชันต่างๆ (web application)

2.1.2 จากจาวา Java Servlet มาเป็น JSP

JSP ย่อมาจากจาวา Java Servlet Page เป็นเทคโนโลยีที่ใช้ “สคริปต์” ในการพัฒนาเว็บแอปพลิเคชัน เพื่อทำงานทางฝั่งเซิร์ฟเวอร์ (server-side script) และส่งผลลัพธ์กลับมายังเว็บเบราว์เซอร์เป็นภาษา HTML เหมือนกับเทคโนโลยีอื่นๆ เช่น ASP, PHP, Cold Fusion เป็นต้น

การเขียนสคริปต์ JSP จะใช้ภาษาจาวาเป็นหลัก ซึ่งเป็นภาษาที่ได้รับความนิยมอย่างมากภาษาหนึ่ง เนื่องจากมีคุณลักษณะทางภาษาเชิงวัตถุที่มีเทคนิคช่วยในการเขียนโปรแกรมได้ง่ายขึ้น มีความสามารถในการนำส่วนประกอบหรือคอมโพเนนต์ต่างๆ (component) กลับมาใช้งานได้อีก ซึ่งเป็นประโยชน์อย่างยิ่งโดยเฉพาะในการพัฒนาโปรแกรมขนาดใหญ่

ทั้ง JSP และ Java Servlet ล้วนพัฒนาขึ้นโดยบริษัทซันไมโครซิสเต็มส์ (Sun Microsystems) เพื่อเอาไว้ใช้สำหรับการสร้างเว็บแอปพลิเคชันด้วยภาษาจาวาเช่นเดียวกัน แต่ข้อแตกต่างระหว่าง JSP กับ จาวาเซิร์ฟเล็ตก็คือ JSP เป็นสคริปต์ ฉะนั้นเวลาพัฒนาเว็บแอปพลิเคชันเราจะสามารถเขียนแต่คำสั่งของ

JSP แทรกลงไปบริเวณที่ต้องการ ภายในเอกสาร HTML ได้ทันที โดยการเขียนแท็กเปิด แล้วตามด้วยซอร์ซโค้ด JSP และปิดด้วยแท็กปิด ในลักษณะเดียวกับสคริปต์ ASP, PHP หรือ JavaScript ตัวอย่างเช่น ถ้าต้องการสร้างเว็บแอปพลิเคชันง่ายๆเพื่อส่งพิมพ์ข้อความ “Java Servlet และ JSP” โดยใช้ JSP จะเขียนสคริปต์ออกมาได้ดังนี้

```
<html>
<body>
    <%
        out.println("Java Servlet และ JSP")
    %>
</body>
</html>
```

ทีนี้ทางเซิร์ฟเล็ตซึ่งเป็นโปรแกรม ไม่ใช่สคริปต์กันบ้าง เราไม่สามารถเขียนเซิร์ฟเล็ตด้วยวิธีแทรกซอร์ซโค้ดเข้าไปโดยตรงแบบเดียวกับ JSP แต่ต้องนำแท็กคำสั่งของเอชทีเอ็มแอล (HTML) ทั้งหมด มาเขียนในลักษณะคลาส (class) ของภาษาจาวา ฉะนั้นจากตัวอย่างเดิมเดิมที่เขียนด้วย JSP ถ้าเราเปลี่ยนมาเขียนเว็บแอปพลิเคชันเป็นจาวาเซิร์ฟเล็ตแทน จะได้ซอร์ซโค้ดดังนี้

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class hello extends HttpServlet
{
    public void doGet (HttpServletRequest request,HttpServletResponse response)
        throws ServletException, IOException
    {
        reponse.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("Java Servlets และ JSP");
        out.priontln("</body></html>");
    }
};
```

เมื่อเปรียบเทียบกันจะเห็นได้ว่าการเขียน JSP ง่ายกว่าจาวาเซิร์ฟเล็ตมาก รวมทั้งการตรวจสอบแก้ไขก็ทำได้สะดวกกว่า ถ้าจะพูดกันตามความเป็นจริงแล้ว JSP ก็คือเซิร์ฟเล็ตที่เขียนออกมาในรูปแบบที่ง่ายขึ้นนั่นเอง สคริปต์ JSP จึงได้รับความนิยมในการใช้สำหรับพัฒนาเว็บแอปพลิเคชันมากกว่าจาวาเซิร์ฟเล็ต

อย่างไรก็ตาม ในขั้นตอนการทำงานจริงนั้น คอมไพเลอร์ก็ต้องแปลง JSP ไปเป็นจาวาเซิร์ฟเล็ตก่อนอยู่ดี จึงจะสามารถทำงานได้

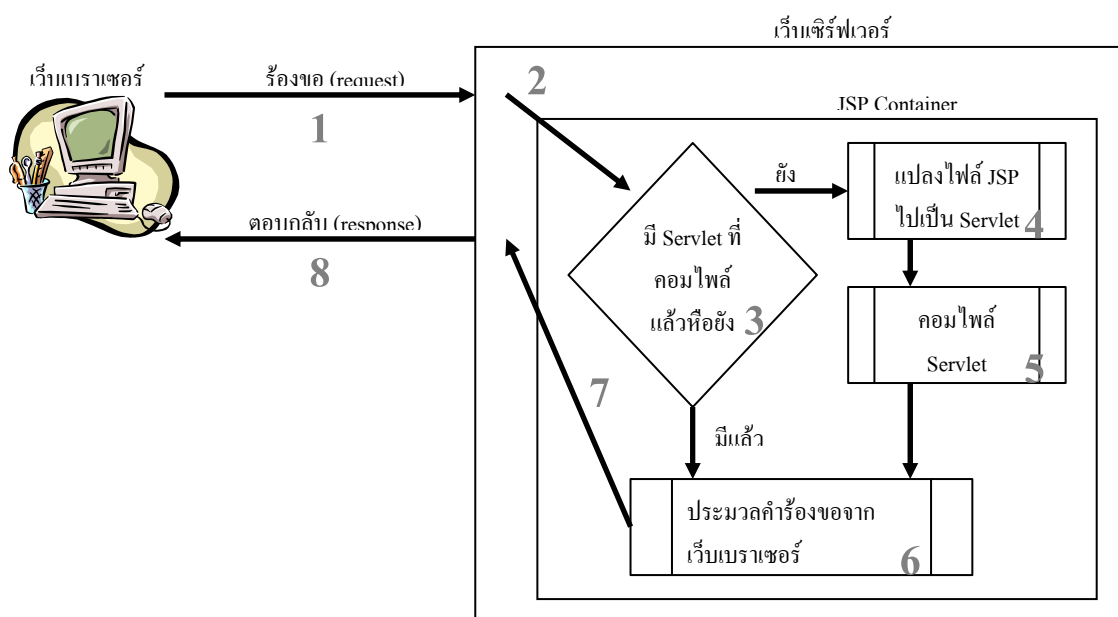
2.1.3 ข้อดีของ JSP

การพัฒนาเว็บแอปพลิเคชันด้วย JSP มีข้อดีต่างๆมากมายดังนี้

- ทำงานโดยไม่ยึดติดแพลตฟอร์มใดๆ คือ JSP ได้สืบทอดคุณสมบัติเด่นของจาวามาอย่างเต็มเปี่ยม คือ การทำงานโดยไม่ขึ้นกับแพลตฟอร์มหรือระบบปฏิบัติการใดๆ ดังนั้นเมื่อพัฒนาในแพลตฟอร์มหนึ่งก็สามารถย้ายไปใช้งานกับแพลตฟอร์มอื่นได้ไม่ยาก
- ใช้งานจาวาเอพีไอ (Java API) ได้หลากหลาย ซึ่งจาวาเอพีไอคือกลุ่มของคลาสที่ช่วยอำนวยความสะดวกในการใช้งานต่างๆ
- นำคอมโพเนนต์กลับมาใช้ได้ อีก ไม่ต้องเสียเวลาสร้างใหม่ คือ เราสามารถนำจาวา빈 (Java Bean) มาใช้งานร่วมกับสคริปต์ JSP เพราะจาวา빈เป็นคอมโพเนนต์ที่เขียนขึ้นมาเพื่อใช้สำหรับทำงานหรือทำหน้าที่อย่างใดอย่างหนึ่ง และสามารถนำกลับมาใช้งานได้เสมอ ดังนั้นเราไม่ต้องเสียเวลาเขียนสคริปต์ JSP เพื่อทำงานนั้นทุกครั้ง จึงพัฒนาเว็บไซต์ได้เร็วขึ้น
- มีความยืดหยุ่นในการใช้งาน คือ ในการเขียนสคริปต์ JSP เราสามารถกำหนดเท็กใหม่ขึ้นมาใช้งานให้เหมาะสมกับความต้องการได้ นอกจากนี้ภาษาที่ใช้เขียนสคริปต์ JSP ไม่ได้จำกัดเฉพาะจาวาเท่านั้น ตามหลักการแล้วเรายังสามารถนำไปใช้งานร่วมกับเอ็กซ์เอ็มแอล (XML) ได้อย่างมีประสิทธิภาพอีกด้วย
- ความปลอดภัย เนื่องจาก JSP มีระบบการจัดการข้อผิดพลาดต่างๆ ไม่ว่าจะเป็นข้อผิดพลาดที่เกิดขึ้นในระหว่างการเขียนสคริปต์ หรือข้อผิดพลาดที่เกิดขึ้นเมื่อนำสคริปต์ JSP ไปใช้งานจริง ล้วนสามารถตรวจสอบและรายงานข้อผิดพลาดได้ทันที

2.1.4 โครงสร้างและขั้นตอนการทำงานของ JSP

สิ่งที่มีบทบาทสำคัญในการทำงานของ JSP ได้แก่ JSP Container (หรือเรียกอีกอย่างหนึ่งว่า JSP Engine) ซึ่งเป็นส่วนประกอบสำคัญที่อยู่ในเว็บเซิร์ฟเวอร์ เพราะทำหน้าที่ควบคุมและประมวลผลไฟล์ JSP ที่มีการร้องขอ (request) เข้ามา และตอบสนอง (response) คำร้องขอนั้นกลับไปยังไคลเอนต์ ขั้นตอนการประมวลผลไฟล์ JSP ทั้งหมดแบ่งเป็น 8 ขั้นตอนดังที่ดังนี้



รูปที่ 2-1 โครงสร้างและขั้นตอนการประมวลผลไฟล์ JSP

- 1) ฟังก์ชันคลเอนต์ส่งคำร้องขอเอกสาร JSP ไปที่เว็บเซิร์ฟเวอร์
- 2) เว็บเซิร์ฟเวอร์ตรวจสอบคำร้องขอ พบว่าเป็นไฟล์ JSP จึงส่งต่อไปให้แก่ JSP Container
- 3) JSP Container ตรวจสอบว่าไฟล์ JSP ที่ร้องขอมาเคยแปลงเป็น Servlet และคอมไพล์เป็นไฟล์ .class แล้วหรือยัง ถ้ายังไม่มีก็จะกระโดดข้ามไปทำงานตามขั้นตอนข้อ 4) ต่อ แต่ถ้ามีอยู่แล้วก็จะตรวจสอบอีกว่า หลังจากแปลงไฟล์เป็น JSP เป็น Servlet และคอมไพล์เป็นไฟล์ .class ครึ่งล่าสุดแล้ว ไฟล์ JSP นั้นมีการเปลี่ยนแปลงแก้ไขหรือไม่ ถ้ามีการแก้ไขก็จะกระโดดไปทำงานตามขั้นตอนข้อ 4) ต่อเช่นกัน แต่ถ้าไม่มีการแก้ไข แสดงว่าไฟล์ JSP นั้นยังคงเดิมไม่เปลี่ยนแปลง จึงไม่มีความจำเป็นที่จะต้องแปลงเป็น Servlet และคอมไพล์ใหม่ ก็ข้ามไปยังขั้นตอนข้อ 6) ได้เลย
- 4) JSP Container แปลงไฟล์ JSP เป็น Java Servlet
- 5) JSP Container คอมไพล์ไฟล์ Java Servlet เป็นไฟล์ .class
- 6) JSP Container ประมวลผลตามคำร้องขอนั้น
- 7) JSP Container ส่งผลลัพธ์ที่ได้จากการประมวลผลให้แก่เว็บเซิร์ฟเวอร์
- 8) เว็บเซิร์ฟเวอร์ส่งผลลัพธ์นั้นไปยังคลเอนต์หรือเว็บเบราว์เซอร์อีกทอดหนึ่ง

จากขั้นตอนการประมวลผลไฟล์ JSP ที่แจกแจงข้างต้น สามารถแบ่งออกได้เป็น 2 ช่วงหลักๆ คือ ช่วง translation และช่วง execution โดยช่วง translation ได้แก่ขั้นตอนข้อ 4) และขั้นตอนข้อ 5) ซึ่ง

เป็นการแปลงเอกสาร JSP (ไฟล์ .jsp) ให้เป็น Servlet (ไฟล์ .java) จากนั้นก็จะคอมไพล์ไฟล์ Servlet ให้เป็น .class ดังแสดงในรูปที่ 3.3.2



รูปที่ 2-2 ขั้นตอนการประมวลผลไฟล์ JSP ในช่วง translation

ส่วนช่วง execution ได้แก่ขั้นตอนข้อ 6) ซึ่งเป็นการนำเอาไฟล์ .class ที่ได้จากการคอมไพล์ มาประมวลผลหรือทำงานตามคำร้องขอจากไคลเอนต์นั่นเอง

ปกติแล้วกระบวนการทำงานในช่วง translation จะกินเวลาพอสมควร แต่โชคดีว่าช่วง translation จะไม่เกิดขึ้นทุกครั้งที่มีการร้องขอไฟล์ เพราะตราบไคที่ไฟล์ JSP ต้นฉบับไม่มีการเปลี่ยนแปลงอะไร เมื่อมีการร้องขอไฟล์เข้ามาใหม่ ก็ย่อมไม่มีความจำเป็นที่จะแปลงไฟล์เป็น Servlet และคอมไพล์เป็นไฟล์ .class อีก ระบบจะเข้าสู่ช่วง execution ทันทีโดยใช้ไฟล์ .class ที่มีอยู่แล้ว การทำงานจึงรวดเร็วยิ่งขึ้น แต่ถ้ามีการเปลี่ยนแปลงหรือแก้ไขไฟล์ JSP ใหม่ ก็จะต้องเข้าสู่กระบวนการ translation ใหม่ทุกครั้ง

สรุปว่ากระบวนการ translation มีโอกาสจะเกิดขึ้นได้ 2 กรณี กรณีแรกคือ ไฟล์ JSP ที่ร้องขอมาเป็นไฟล์ใหม่ที่ยังไม่เคยแปลงหรือคอมไพล์มาก่อน กับอีกกรณีคือไฟล์ JSP ที่ร้องขอมาเคยผ่านการแปลงและคอมไพล์มาแล้ว แต่ภายหลังจากนั้นมีการเปลี่ยนแปลงแก้ไขไฟล์ JSP นั้นไปจากเดิม

2.2 Java Applet

2.2.1 เบื้องต้น

Applet คือโปรแกรมขนาดเล็กที่สร้างขึ้นด้วยภาษาจาวา สามารถถูกเรียกจากใน HTML page ให้ทำงานเป็นส่วนหนึ่งของเว็บเพจนั้น กล่าวอีกนัยหนึ่งได้ว่าเป็นโปรแกรมที่ถูกส่งไปกับ HTML page เพื่อไปทำงานภายใต้เบราว์เซอร์ที่มีตัวแปลภาษาจาวา (Java Interpreter) บนเครื่องที่รับ HTML page นั้นๆ ไปสังเกตว่าภาษาจาวาที่เป็นแอปพลิเคชันจะทำงานภายใต้ Java Interpreter โดยไม่จำเป็นต้องอาศัยโปรแกรมอื่น (จึงเรียกว่าเป็น standalone โปรแกรม) และสามารถควบคุมการดำเนินงานของตัวเองได้ แต่โปรแกรมที่เป็น applet จะทำงานภายใต้เบราว์เซอร์โดยไม่สามารถควบคุมการดำเนินงานของตัวเองได้ทั้งหมด

คลาส Applet เก็บอยู่ในแพ็คเกจ `java.applet` ที่มากับ JDK (Java Development Kit) ใช้สำหรับสร้าง applet ทุกโปรแกรมของ applet ต้องมีประโยค `import java.applet.*;` เพื่อให้มองเห็นคลาส applet และประโยค `import java.awt.*;` เพื่อให้คอมไพเลอร์มองเห็นแพ็คเกจที่เรียกว่า Abstract Window Toolkit (AWT) ซึ่งเป็นที่รวบรวมของคลาสเกี่ยวกับกราฟิกและ GUI (Graphic User Interface) โดยทั่วไปเราจะไม่สร้าง instance ของคลาส applet ขึ้นใช้โดยตรง เพราะจะได้ applet ที่ไม่สามารถทำอะไรได้เลย แต่จะใช้งานคลาส Applet โดยขยายไปเป็นคลาสลูก โดยทำการ override methods บางตัวให้ applet ที่ได้

นั้นทำงานตามที่เรากำลังต้องการ สังเกตว่าโปรแกรมของ applet ไม่จำเป็นต้องมี main() เป็นจุดเริ่มต้นการทำงาน (มีก็ได้แต่จะไม่ถูกเรียกใช้) เพราะ applet มีช่วงชีวิตที่ถูกควบคุมโดยเบราว์เซอร์

โปรแกรมของ applet เมื่อถูกคอมไพล์แล้วจะได้ไฟล์เป็น .class (อาจจะมีหลายไฟล์ขึ้นอยู่กับวิธีการเขียน) แต่จะไม่สามารถทำงานได้โดย Java Interpreter (Java.exe) การเรียกให้ applet ทำงานนั้นต้องทำงานจากภายใน HTML page โดยการใช้ Applet Tag และ applet นั้นจะทำงานภายในสภาพแวดล้อมของเบราว์เซอร์ อย่างเช่น Netscape Navigator , Microsoft Internet Explorer หรือ Hot Java ซึ่งทั้งหมดนี้เป็นเบราว์เซอร์ที่มี Java Interpreter อยู่ภายใน หรือ โปรแกรมสำหรับใช้ทดสอบ applet ที่มากับ JDK หรือ appletviewer

ตัวอย่างการสร้าง applet โดยมีโปรแกรมของ applet อยู่ในไฟล์ HelloA.java ดังนี้

```
import java.awt.*;
import java.applet.*;

public class HelloA extends Applet
{
    public void paint (Graphics g)
    {
        g.drawString("Hello",25,50);
    }
}
```

เมื่อโปรแกรมถูกคอมไพล์แล้วจะได้ไฟล์ HelloA.class นอกจากนี้จะต้องมี HTML Page ที่มี Applet Tag ที่เรียกว่า HelloA.class ให้ทำงานอยู่ในไฟล์ HelloA.html ดังนี้

```
<html>
<body>
    This is an Applet.
    <applet code="HelloA.class" width=200 height=200></applet>
</body>
</html>
```

2.2.2 คลาส Applet

คลาส applet จะถูกกำหนดไว้ใน java.applet.apckage ดังนี้

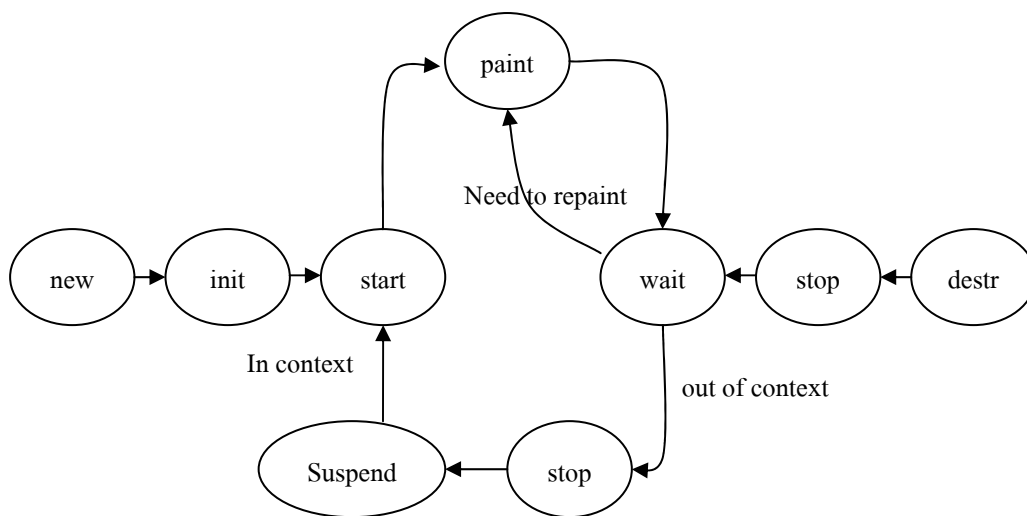
Public class Applet extends Panel

```
{
    public void destroy();
    public AppletContext getAppletContext();
    public String getAppletInfo();
    public AudioClip getAudioClip(URL url);
    public AudioClip getAudioClip(URL url,String name);
    public URL getCodeBase();
    public URL getDocumentBase();
    public Image getImage(URL url);
    public Image getImage(URL url,String name);
    public Locale getLocale(); //Overrides Component
    public String getParameter(String name);
    public String[][] getParameterInfo();
    public void init();
    public Boolean isActive();
    public void play(URL url);
    public void play(URL url,String name);
    public void resize(int width,int height); // Override Component
    public void resize(Dimension d)l // Overrides Component
    public final void setStub(AppletStub stub);
    public void showStatus(String msg);
    public void start();
    public void stop();
}
```

2.2.3 Applet Life Cycle

ช่วงชีวิตของ applet แบ่งออกเป็นสถานะต่างๆ ได้ดังนี้ init, start, paint, wait, suspended, stop และ destroy ทุกๆ applet จะดำเนินการผ่านสถานะเหล่านี้ โดยบางส่วนจะเป็นไปอย่างอัตโนมัติและบางส่วนอยู่ในการควบคุมของผู้ใช้ที่เรีกยอ applet นั้น ช่วงชีวิตของ applet แสดงเป็นรูปการเปลี่ยน

สถานะได้ดังนี้



รูปที่ 2-3 แสดงการเปลี่ยนสถานะของ applet

จากรูปเมื่อเริ่มต้น applet ถูกสร้างโดยการ new() เหมือน instances ทั่วไป ก็จะเริ่มทำงานทันทีคือ

1. เข้าสู่สถานะ init แล้วผ่านเข้าสู่สถานะ start จากนั้นจะเข้าสู่สถานะ paint และเข้าสู่สถานะ wait ซึ่งจะหยุดรอเหตุการณ์ (Event) หรือสัญญาณจากผู้ใช้ (คืออินพุตที่รับเข้าไปทางเมาส์, คีย์บอร์ด หรือ GUI) ที่จะเข้ามาให้เกิดการทำงานของ method ที่กำหนดไว้สำหรับจัดการกับเหตุการณ์นั้น เมื่อเสร็จแล้วจะกลับเข้าสู่สถานะ wait เพื่อรอเหตุการณ์อื่นต่อไป
2. ระหว่างที่ applet อยู่ในสถานะ wait อยู่ หากผู้ใช้เปลี่ยนแปลงเว็บเพจจนทำให้
 - a. พื้นที่แสดงผลของ applet นั้นต้องมีการถูกวาดใหม่ เบราเซอร์จะออกคำสั่งให้ applet ทำการวาดอีกครั้งโดยเรียก repaint() ซึ่งจะส่งผลให้ applet เข้าสู่สถานะ paint และทำการวาดพื้นที่แสดงผลของ applet นั้นอีกครั้ง เมื่อเสร็จแล้วจะกลับเข้าสู่สถานะ visit เพื่อรอเหตุการณ์อื่นต่อไป
 - b. พื้นที่แสดงผลของ applet นั้นไม่ปรากฏในเว็บเพจปัจจุบัน จะทำให้เบราเซอร์ออกคำสั่งให้ applet ผ่านเข้าสู่สถานะ stop และไปสู่สถานะ suspended ซึ่ง applet จะหยุดรอในสถานะนี้โดยไม่ยอมรับอินพุตจากผู้ใช้ และไม่ได้ตอบใดๆจนกว่าพื้นที่แสดงผลของ applet นั้นจะถูกทำให้กลับมาอีกครั้ง เบราเซอร์ก็จะออกคำสั่งให้ applet เข้าสู่สถานะ start, paint และ wait เพื่อเริ่มทำงานอีก
3. ระหว่างที่ applet อยู่ในสถานะ wait หากผู้ใช้กดปุ่มปิดของเบราเซอร์เพื่อหยุดการทำงาน จะทำให้ applet เข้าสู่สถานะ stop และไปสู่สถานะ destroy แล้วสิ้นสุดการทำงาน

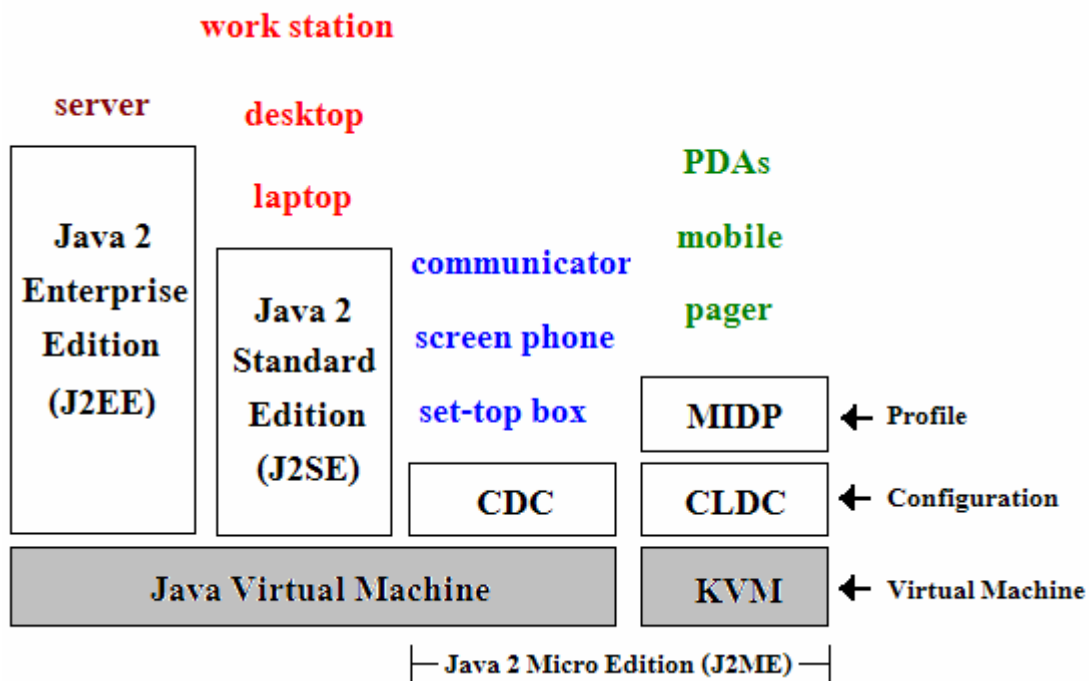
ในการสร้างโปรแกรมของ applet เราสามารถกำหนดรูปแบบการทำงานในแต่ละสถานะต่างๆที่กล่าวมา เมื่อ applet ดำเนินการผ่านคลาส Applet ก็จะมี methods ชื่อเหมือนกับสถานะคือ init(), start(), paint(), stop() และ destroy() สำหรับกำหนดการทำงานเมื่อตอน applet นั้นเข้าสู่สถานะนั้นๆ ถ้าตรวจสอบดูจะพบว่า paint() ไม่ได้ถูกกำหนดไว้ในคลาส applet แต่ได้รับการถ่ายทอดมาจากคลาส Component และสังเกตว่าจะไม่มี method สำหรับสถานะ suspend และ wait เนื่องจากการเข้าและออกจากสถานะ suspended นั้นอยู่ในการควบคุมของเบราเซอร์

2.3 J2ME

2.3.1 เบื้องต้น

J2ME (Java 2 Micro Edition) เป็นภาษาจาวาเวอร์ชันปัจจุบันที่พัฒนาโดยบริษัท Sun Microsystems ออกแบบเพื่อติดตั้ง ใช้งานกับแอปพลิเคชันในอุปกรณ์ที่มีขนาดเล็กและมีทรัพยากรของระบบจำกัด เช่น โทรศัพท์มือถือและคอมพิวเตอร์ขนาดเล็กแบบพกพา (Personal Digital Assistants) เป็นต้น J2ME เป็นภาษาเชิงวัตถุที่มีจุดเด่นตรงที่สามารถนำไปรันในทุกๆ ระบบปฏิบัติการได้ เพราะในแพลตฟอร์มของ J2ME จะมี Java Virtual Machine ที่ทำหน้าที่แปลความหมายไบต์โค้ดหรือคลาสจาวาที่ถูกคอมไพล์แล้วไปเป็นภาษาเครื่องของแต่ละระบบปฏิบัติการอีกที

J2ME เป็นภาษาจาวาที่มีขนาดเล็กที่สุด เพื่อที่จะสามารถติดตั้งบนอุปกรณ์ขนาดเล็กได้ กลุ่มคลาสของ J2ME แบ่งได้เป็น 2 กลุ่มโดยกลุ่มแรกเป็นคลาสที่พัฒนาขึ้นมาใหม่ และกลุ่มที่สองเป็นคลาสบางส่วนที่นำมาจาก J2SE สาเหตุที่ไม่สามารถนำคลาสทั้งหมดของ J2SE มาใช้งานได้เพราะคลาสบางตัวต้องใช้ทรัพยากรของระบบมากและมีรูปแบบที่ไม่เหมาะสมกับอุปกรณ์ เช่น คลาส AWT (Abstract Window Toolkit) ซึ่งออกแบบมาเพื่อใช้งานกับระบบปฏิบัติการวินโดวส์ เป็นต้น



รูปที่ 2-4 แสดงการเปรียบเทียบภาษาจาวาเวอร์ชันต่างๆ J2EE (Java 2 Enterprise Edition) ทำงานบนเครื่องเซิร์ฟเวอร์ใช้สำหรับองค์กรที่รองรับระบบงานใหญ่ และมีไคลเอนต์จำนวนมาก J2SE (Java 2 Standard Edition) ทำงานบนเครื่องคอมพิวเตอร์ประเภทเดสก์ท็อปและเวิร์กสเตชัน และ J2ME (Java 2 Micro Edition) เป็นภาษาจาวาเวอร์ชันเล็กที่สุด ทำงานบนอุปกรณ์ประเภทมือถือ (Hand-Held Devices) เช่น โทรศัพท์มือถือและคอมพิวเตอร์ขนาดพกพา (Personal Digital Assistants) สถาปัตยกรรม J2ME และ MIDP

Profile	MID Profile
Configuration	CLDC
Java Virtual Machine	K Virtual Machine
Host Operating System	Host Operating System

รูปที่ 2-5 แสดงสถาปัตยกรรม J2ME และสถาปัตยกรรม MIDP

2.3.2 แพลตฟอร์มของจาวาโทโมบายส์อีดิชัน

จะแบ่งเป็น 4 ส่วนดังนี้

1) **Profile** เป็นกลุ่มคลาสไลบรารีที่รองรับข้อแตกต่างของอุปกรณ์ เช่น Profile ของอุปกรณ์ประเภทมือถือคือ MIDP (Mobile Information Device Profile) จะเป็นกลุ่มคลาสไลบรารีต่างๆที่รองรับ

การพัฒนาแอปพลิเคชันบนอุปกรณ์ประเภทมือถือ ประกอบด้วย UI (User Interface), Input/Output, Event Handling, Persistent Storage, Networking และ Timers

2) Configuration เป็นกลุ่มคลาสไลบรารีพื้นฐานสำหรับอุปกรณ์ประเภทมือถือทุกประเภท การที่จะให้กลุ่มคลาสเหล่านี้ครอบคลุมทุกอุปกรณ์นั้น จำเป็นต้องนำอุปกรณ์มาจัดกลุ่มโดยใช้คุณสมบัติพื้นฐานของอุปกรณ์ คือ หน่วยความจำ การแสดงผล การเชื่อมต่อสัญญาณ และการใช้พลังงาน ปัจจุบัน Configuration ใน J2ME แบ่งเป็น 2 ประเภท คือ Connect Device Configuration (CDC) เป็นอุปกรณ์ใช้ร่วมกับอุปกรณ์อื่น ติดตั้งตายตัวและใช้เชื่อมต่อข้อมูล โดยปรกติมักมีส่วนติดต่อผู้ใช้หลากหลายแบบ มีหน่วยความจำประมาณ 2 – 16 เมกกะไบต์ ตัวอย่างอุปกรณ์ประเภทนี้ได้แก่ โทรศัพท์มือถือขนาดเล็ก โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต อุปกรณ์สื่อสารที่มีความซับซ้อนสูง และ Connected, Limited Device Configuration (CLDC) ซึ่งเป็นกลุ่มคลาสอุปกรณ์ส่วนบุคคล พกพาได้ และใช้เชื่อมต่อข้อมูล โดยปรกติจะมีส่วนติดต่อผู้ใช้อย่างง่าย มีหน่วยความจำประมาณ 128 กิโลไบต์ – 1 เมกกะไบต์ ตัวอย่างอุปกรณ์ประเภทนี้ได้แก่ โทรศัพท์มือถือแบบไม่ซับซ้อนมากนัก เพจเจอร์รับส่งข้อความ เครื่องคอมพิวเตอร์แบบพกพา

3) Java Virtual Machine โปรแกรมภาษาทุกตัวจะทำงานภายใต้ JVM เสมอโดยเมื่อเราคอมไพล์โปรแกรมจาวาเป็นคลาสแล้ว JVM จะทำหน้าที่แปลงคลาสนั้นไปเป็นภาษาเครื่องและทำงานต่อไป ใน J2ME จะใช้ Configuration เป็นตัวกำหนด JVM โดยกลุ่ม CDC ใช้กลุ่มคลาสชุดเดียวกันกับ J2SE ทำให้สามารถใช้ JVM ได้แต่สำหรับกลุ่ม CLDC ไม่ได้ใช้กลุ่มคลาสของ J2SE ทั้งหมดทำให้ต้องพัฒนา JVM ขึ้นมาใหม่ เรียกว่า K Virtual Machine (KVM) ซึ่งมีคุณสมบัติเป็น JVM ที่ใช้งานในอุปกรณ์ประเภทมือถือ

4) Host Operating System คือ ระบบปฏิบัติการของอุปกรณ์แต่ละตัว

2.3.3 MIDP

MIDP คือ Profile สำหรับพัฒนาแอปพลิเคชันบนอุปกรณ์ประเภทมือถือ ซึ่งในที่นี้เราจะใช้เวอร์ชัน 1.0 และใช้ KVM เป็น Virtual Machine แทน JVM MIDP ประกอบด้วยแพ็คเกจทั้งหมด 7 แพ็คเกจ ดังนี้

แพ็คเกจ	คำอธิบาย
java.io	กลุ่มคลาสที่มาจาก J2SE สำหรับส่งข้อมูล (Input/Output)
java.lang	กลุ่มคลาสที่มาจาก J2SE ที่เกี่ยวกับจาวา
java.util	กลุ่มคลาสที่มาจาก J2SE ประกอบด้วยคลาสยูทิลิตี้ต่างๆ
javax.microedition.io	กลุ่มคลาสสำหรับรับส่งข้อมูลผ่านระบบเน็ตเวิร์ก
javax.microedition.lcdui	กลุ่มคลาสของคอมโพเนนต์ต่างๆ
javax.microedition.midlet	กลุ่มคลาสสำหรับแอปพลิเคชันของ J2SE
javax.microedition.rms	กลุ่มคลาสสำหรับจัดการข้อมูลบนหน่วยความจำถาวรของอุปกรณ์

ตารางที่ 2-1 แสดง packet ของ JMP MIDP